ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

RAPPORT DE PROJET

SMART GRADES

Module : Python Avancé

Membres du groupe :

HARDOUMI Ismail

ATIF Yassine

ETTAOUIL Abdelilah

Sous la supervision par :

Pr. HAIN Mustapha

Table des matières

1. INTRODUCTION ____ *2*

1. Contexte et Problématique ____ **2**

2. Fonctionnalités Principales ____ **2**

2. CONCEPTION DE LA BASE DE DONNÉES ____ *2*

1. Philosophie et Choix Technologiques ____ **2**

2. Modélisation et Schéma Relationnel ____ **2**

Relations et Contraintes d'intégrité : ____ 3

3. ARCHITECTURE LOGICIELLE ET FLUX D'EXÉCUTION ____ *3*

1. Organisation du Code ____ **3**

2. Flux d'exécution et Séparation des responsabilités ____ **3**

4. PRÉSENTATION DES INTERFACES GRAPHIQUES (Tkinter) ____ *4*

1. Fenêtre d'authentification / Accueil ____ **4**

2. Interface principale de gestion (Espace Professeur) ____ **4**

3. Fenêtres secondaires, tableaux et alertes (Espace Étudiant) ____ **7**

5. Extraits de Code Remarquables ____ *7*

Extrait 1 : Requête SQL sécurisée avec syntaxe Upsert ____ 7

Extrait 2 : Gestion des événements Tkinter et rafraîchissement d'interface ____ 8

6. Conclusion et Perspectives ____ *8*

1. Bilan des compétences validées ____ **8**

2. Difficultés surmontées ____ **8**

3. Perspectives d'évolution ____ **8**

1. INTRODUCTION

1. Contexte et Problématique

Dans le cadre de la digitalisation des établissements d'enseignement, la gestion des notes et du suivi des étudiants reste un enjeu majeur. De nombreuses solutions existent sur le marché, mais elles sont souvent lourdes à déployer ou complexes à prendre en main pour les équipes pédagogiques. Ce projet s'inscrit donc dans la volonté de créer un outil local, rapide et simple à prendre en main.

La problématique abordée est la suivante : Comment permettre à un corps professoral de saisir et de modifier des notes de manière sécurisée tout en offrant aux étudiants un espace unifié pour consulter leurs résultats, le tout sans dépendre d'une infrastructure serveur complexe ?

L'application *SmartGrades* répond à ce besoin en cloisonnant strictement les espaces de travail selon le rôle défini de l'utilisateur (enseignant ou étudiant).

2. Fonctionnalités Principales

Pour répondre à cette problématique, notre application déploie les fonctionnalités suivantes :

- **Authentification sécurisée :** Connexion par rôle avec hachage cryptographique des mots de passe (algorithme PBKDF2), garantissant qu'aucun mot de passe n'est stocké en clair.
- **Espace Professeur (Édition) :** Interface permettant d'assigner des groupes aux étudiants, de consulter la liste des étudiants à un module spécifique, et de saisir ou modifier les notes (Contrôle continu et Examen).
- **Espace Étudiant (Consultation) :** Tableau de bord dynamique récapitulant les modules, les notes obtenues, les moyennes par module, ainsi que le calcul automatisé de la moyenne pondérée du semestre.

2. CONCEPTION DE LA BASE DE DONNÉES

1. Philosophie et Choix Technologiques

Pour maintenir l'application légère et performante, nous avons opté pour SQLite3. Intégré nativement à Python, ce SGBD ne nécessite aucune configuration réseau. La sécurité et la haute intégrité des données sont assurées par l'activation explicite des contraintes relationnelles via la commande SQL PRAGMA foreign_keys = ON; lors de l'initialisation, garantissant qu'aucune donnée orpheline ne puisse être insérée.

2. Modélisation et Schéma Relationnel

L'architecture des données a été volontairement optimisée pour éviter toute redondance. Elle est restreinte à exactement **3 tables principales** :

- **Table users :** Gère les comptes. Contient l'identifiant (clé primaire), le nom d'utilisateur, le mot de passe haché, le rôle restrictif (student ou teacher), le nom complet, l'email et le nom du groupe (G1, G2...) pour les étudiants.
- **Table modules :** Gère les matières. Contient l'identifiant, le nom du module, le coefficient, et l'ID du professeur responsable (clé étrangère pointant vers users).
- **Table grades :** Gère l'association entre un étudiant, un module et ses notes. Contient les notes de contrôle continu et d'examen.

Relations et Contraintes d'intégrité :

- grades (student_id) ➔ Clé étrangère vers users (user_id)
- grades (module_id) ➔ Clé étrangère vers modules (module_id)
- modules (teacher_id) ➔ Clé étrangère vers users (user_id)
- grades (student_id, module_id) ➔ Contrainte UNIQUE : un seul enregistrement de notes par couple étudiant/module

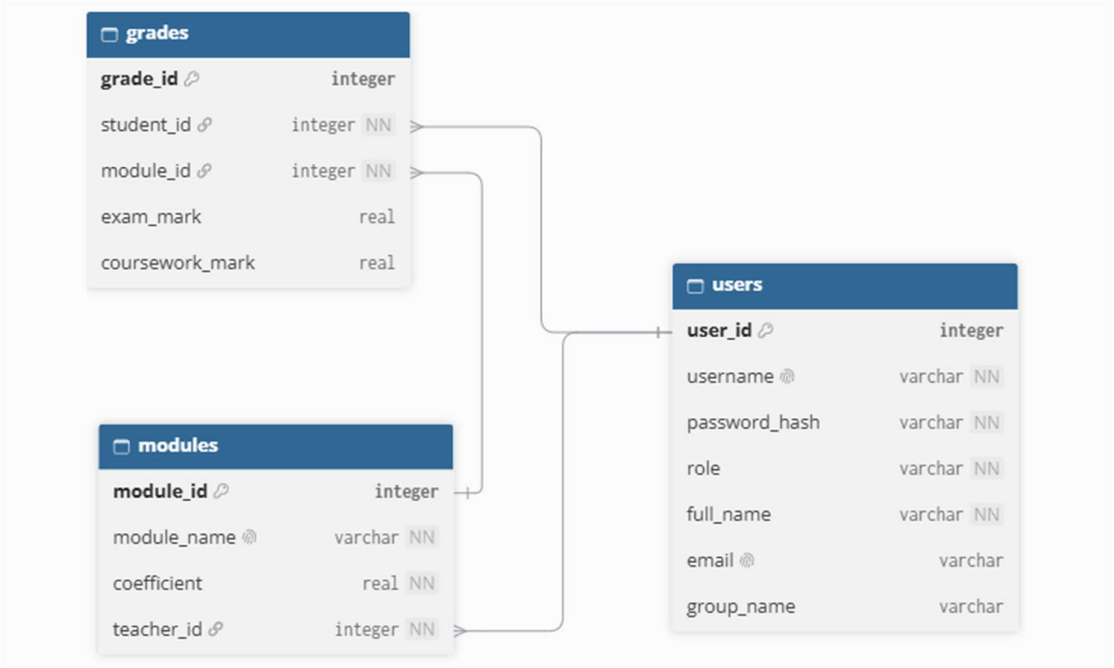


FIGURE 1 : SCHÉMA RELATIONNEL DE LA BASE DE DONNÉES

3. ARCHITECTURE LOGICIELLE ET FLUX D'EXÉCUTION

1. Organisation du Code

Pour garantir la lisibilité et la maintenabilité du code, nous avons adopté une architecture logicielle en couches. Contrairement à un modèle MVC strict, cette approche sépare les responsabilités en isolant l'accès aux données, la logique métier et l'interface utilisateur :

- **main.py** : Point d'entrée de l'application. Il initialise la BDD et lance l'UI.
- **database.py & schema.sql** : Gestion de la connexion SQLite et création des tables de haute intégrité.
- **models/** : Contient les classes Python représentant la structure des entités de données.
- **repositories/** : Couche d'isolation des requêtes SQL.
- **services/** : Contient la logique métier (calcul des moyennes, vérification des mots de passe, règles de validation des notes de 0 à 20).
- **ui/main_window.py** : Contient exclusivement la logique d'interface graphique Tkinter.

2. Flux d'exécution et Séparation des responsabilités

L'amorçage du système est géré par main.py qui exécute initialize_database() (création via schema.sql). Des données de démonstration peuvent être injectées séparément via create_demo_users.py.

Le point fort de cette architecture est la séparation stricte entre l'interface et la logique de données. L'interface Tkinter ne communique jamais directement avec la base de données.

- *Exemple :* Lors de la connexion, main_window.py capture les saisies et délègue la vérification en appelant authenticate() du fichier auth_service.py. C'est cette couche métier qui interroge la base et valide le hachage. Selon le rôle retourné (student ou teacher), le routage dynamique intervient : l'interface détruit la page de connexion initiale et construit dynamiquement le tableau de bord approprié.

4. PRÉSENTATION DES INTERFACES GRAPHIQUES (Tkinter)

1. Fenêtre d'authentification / Accueil

Description : Les éléments du formulaire sont positionnés via le gestionnaire de géométrie `.grid()`. Avant de construire cette vue, la méthode `self.clear()` détruit l'ensemble des widgets enfants précédemment affichés à l'écran. Cette opération assure un nettoyage graphique complet pour éviter les superpositions d'interfaces lors des déconnexions. Le champ mot de passe est masqué visuellement par `show="*"`.

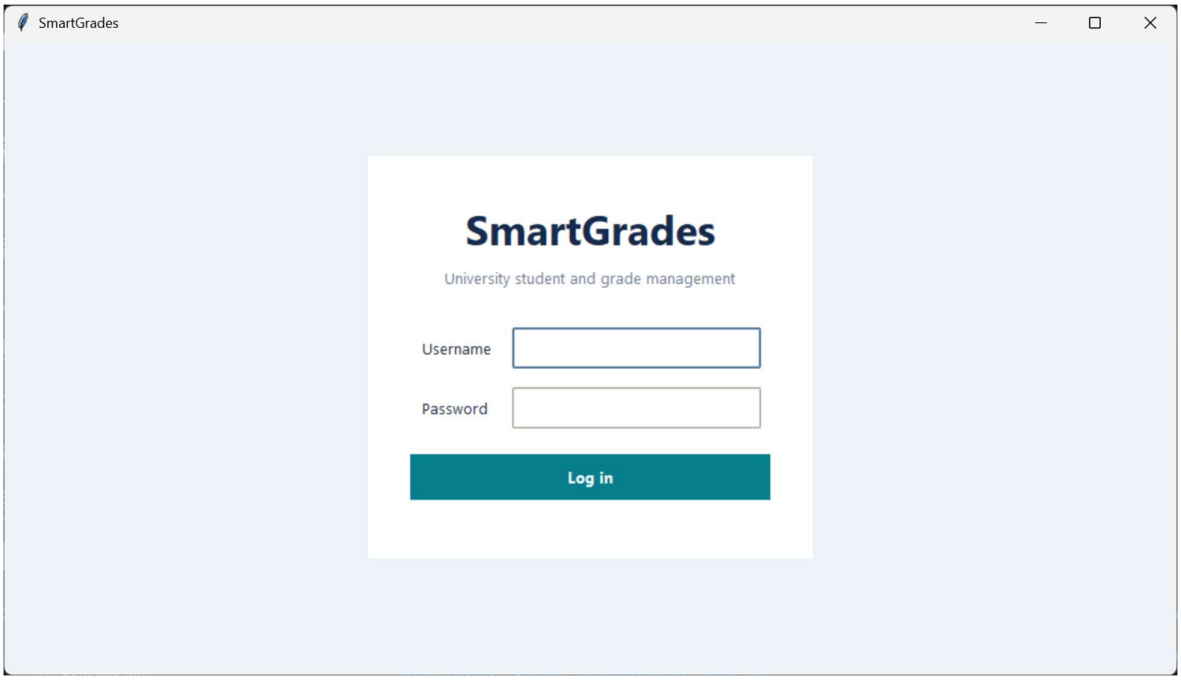


FIGURE 2 : FENÊTRE D'AUTHENTIFICATION ET DE ROUTAGE

2. Interface principale de gestion (Espace Professeur)

Description : Cette interface complexe utilise un composant **`ttk.Notebook`** pour gérer la navigation multi-onglets (Notes, Modules, Étudiants) sans surcharger l'écran. La sélection d'un module ou d'un étudiant est sécurisée via des menus déroulants (**`ttk.Combobox`**) empêchant la saisie manuelle erronée. Un tableau de données dynamique (**`ttk.Treeview`**) affiche en temps réel les élèves inscrits au module et leurs notes actuelles.

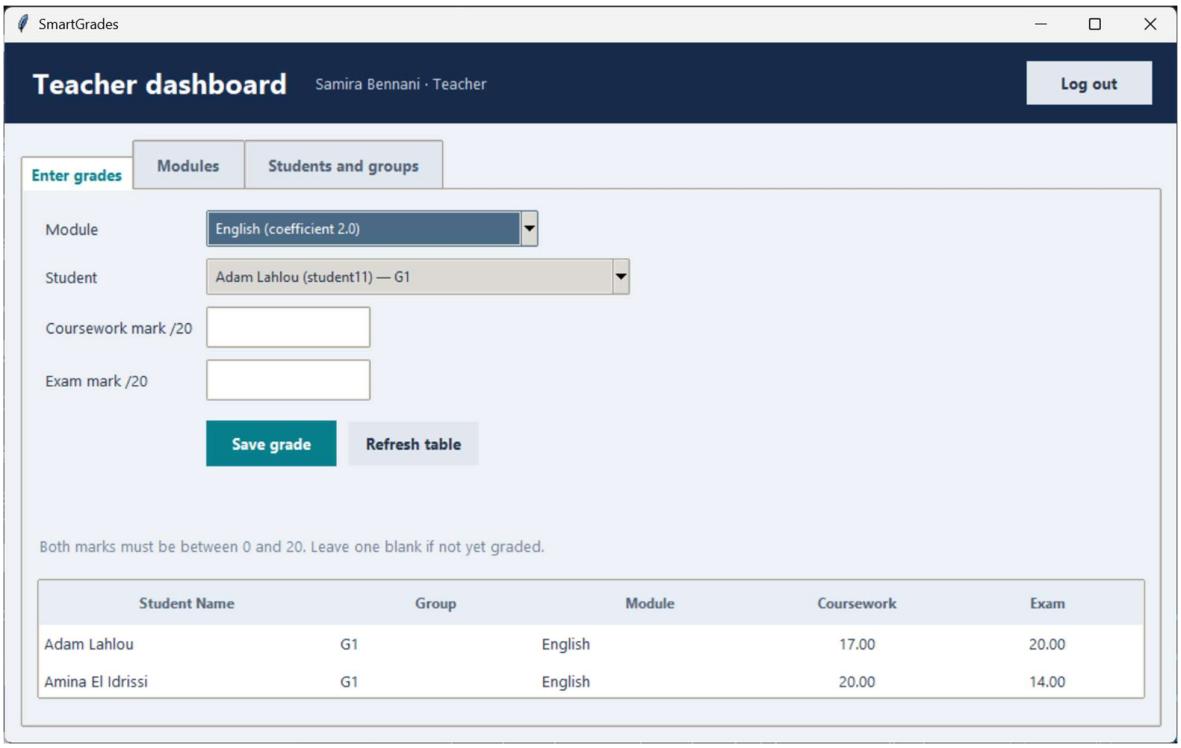


FIGURE 3 : TABLEAU DE BORD PROFESSEUR - SAISIE ET MODIFICATION DES NOTES

Description : Cet onglet présente la liste des modules attribués à l'enseignant connecté. Grâce à une sous-requête SQL optimisée utilisant la fonction d'agrégation `AVG()`, l'application calcule et affiche dynamiquement la moyenne générale de la classe ('Average of class') pour chaque module, en appliquant la pondération 40% contrôle continu et 60% examen.

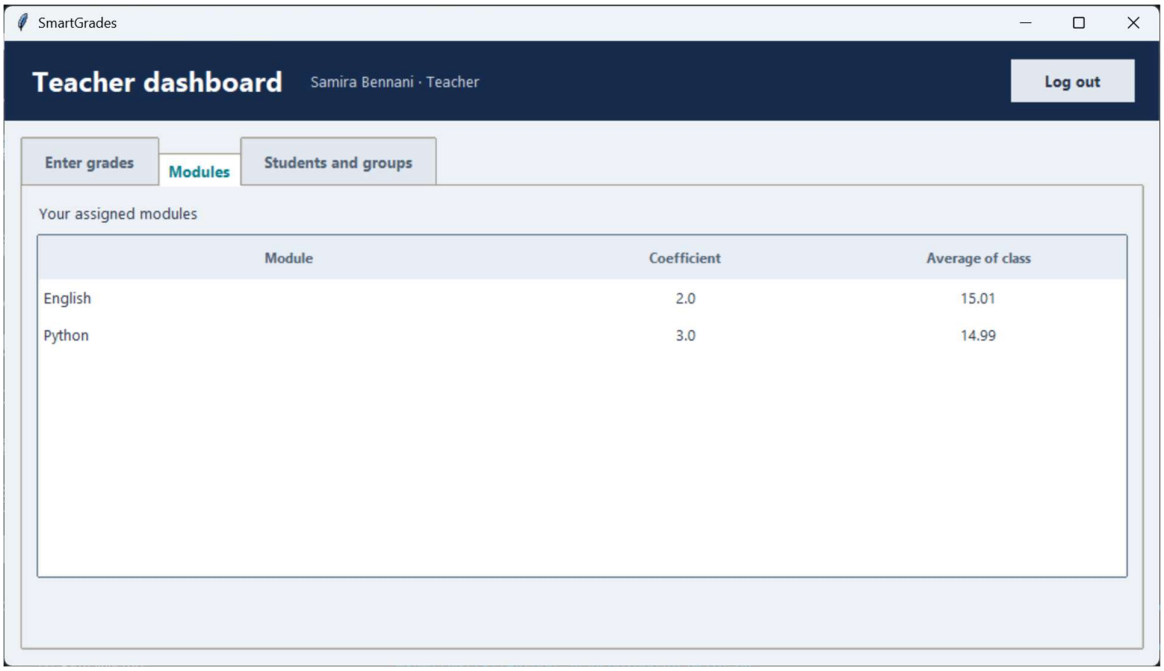


FIGURE 4 : TABLEAU DE BORD PROFESSEUR - CONSULTATION DES MODULES ET MOYENNE DE CLASSE

Description : Cet écran affiche l'ensemble des étudiants inscrits au sein d'un `ttk.Treeview`. En sélectionnant une ligne, le professeur peut saisir un nouveau code de groupe dans la zone d'entrée inférieure (`ttk.Entry`) et valider l'affectation via le bouton 'Assign group', déclenchant une mise à jour immédiate en base de données et sur le tableau.

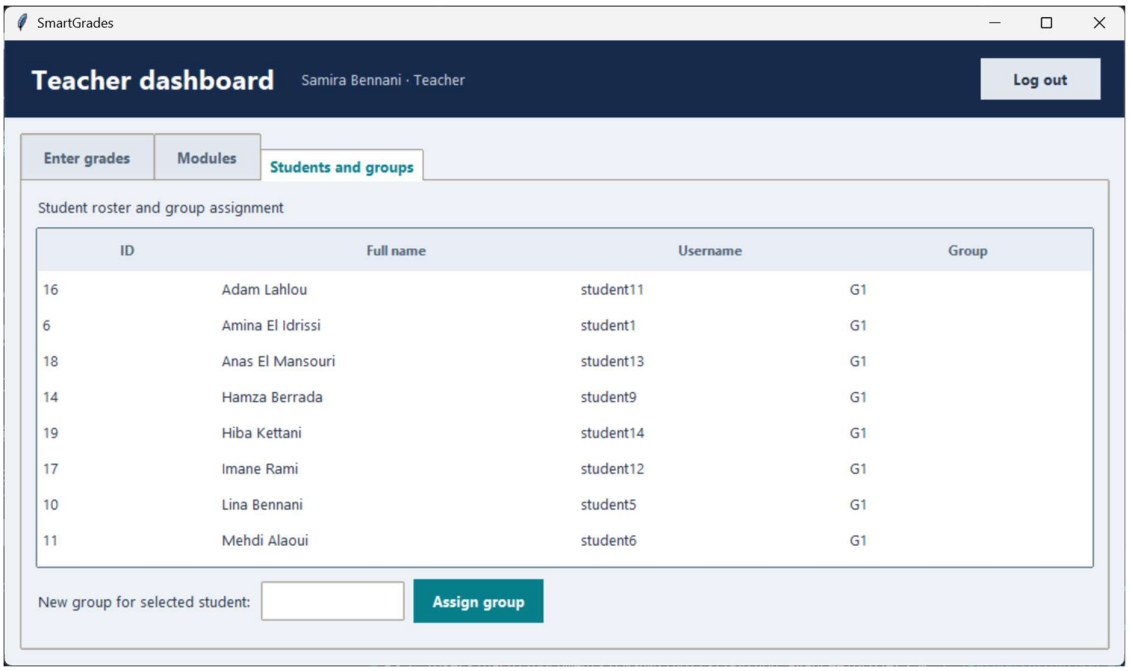


FIGURE 5 : TABLEAU DE BORD PROFESSEUR - LISTE DES ETUDIANTS ET AFFECTATION DES GROUPES

3. Fenêtres secondaires, tableaux et alertes (Espace Étudiant)

Description : L'étudiant visualise ses résultats via un **`Treeview`** formaté en lecture seule. La moyenne semestrielle pondérée est calculée à la volée par les services Python et affichée en évidence. L'interface utilise le système d'alertes natif (**`messagebox.showerror`** ou **`showinfo`**) pour communiquer avec l'utilisateur (ex: confirmation d'une action réussie ou d'un accès refusé).

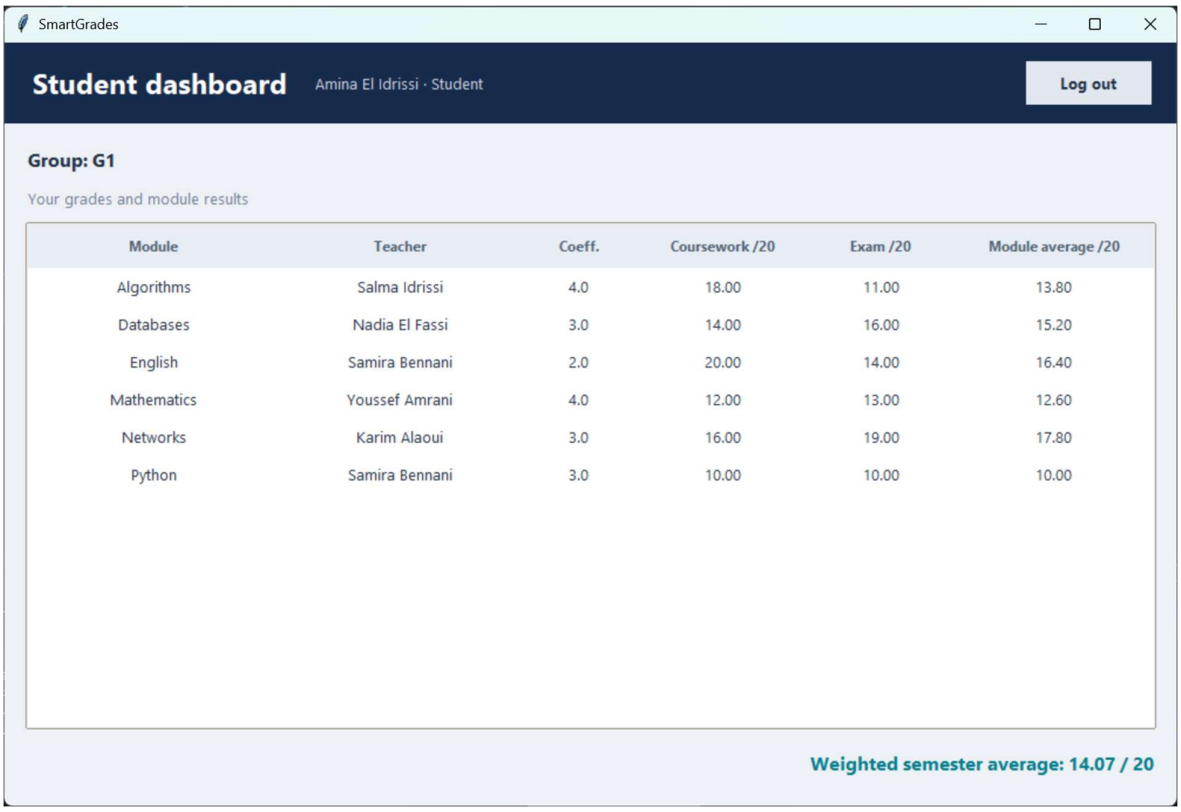


FIGURE 6 : TABLEAU DE BORD ÉTUDIANT - CONSULTATION DES RÉSULTATS SEMESTRIELS

5. Extraits de Code Remarquables

Extrait 1 : Requête SQL sécurisée avec syntaxe Upsert

Ce code illustre la sauvegarde robuste d'une note. Nous utilisons l'instruction avancée SQLite ON CONFLICT DO UPDATE (Upsert) combinée à des requêtes paramétrées (?, ?, ?, ?) pour bloquer toute tentative d'injection SQL.

Fichier : *services/grade_service.py* (lignes 43 à 50)

```
conn.execute(
    """INSERT INTO grades (student_id, module_id, coursework_mark, exam_mark)
       VALUES (?, ?, ?, ?)
       ON CONFLICT(student_id, module_id) DO UPDATE SET
         coursework_mark=excluded.coursework_mark,
         exam_mark=excluded.exam_mark""",
    (student_id, module_id, coursework, exam),
)
```

Analyse : L'utilisation des points d'interrogation (?) garantit que les variables sont traitées comme des valeurs pures et non comme du code exécutable. La clause avancée ON CONFLICT DO UPDATE (Upsert) détecte si une note existe déjà pour ce couple étudiant/module (grâce à la

contrainte UNIQUE de la table). Si elle existe, la requête bascule automatiquement d'une insertion à une modification, évitant ainsi les doublons et les erreurs d'intégrité de manière atomique.

Extrait 2 : Gestion des événements Tkinter et rafraîchissement d'interface

Ce bloc illustre la capture d'un événement graphique sur le Treeview et la mise en œuvre d'une gestion stricte des exceptions Python.

Fichier : *ui/main_window.py* (lignes 333 à 344)

```
def assign_group():
    selected = tree.selection()
    if not selected:
        messagebox.showinfo("Select student", "Select a student row first.")
        return
    try:
        student_id = int(selected[0]) # Risque de ValueError ou IndexError
        update_student_group(student_id, group_var.get())
        tree.set(selected[0], "group", group_var.get().strip())
        messagebox.showinfo("Updated", "Student group updated.")
    except Exception as exc:
        messagebox.showerror("Update failed", str(exc))
```

Analyse : La fonction assign_group() vérifie la sélection de l'utilisateur dans le Treeview. Après avoir extrait l'ID caché (selected[0]), elle fait appel au service métier pour mettre à jour la base de données. L'optimisation majeure ici est l'utilisation de tree.set(...) : l'affichage graphique est mis à jour localement et instantanément, sans obliger le programme à recharger intégralement l'interface ou à refaire une lourde requête SQL de sélection.

6. Conclusion et Perspectives

1. Bilan des compétences validées

Le projet SmartGrades a permis de valider des compétences clés en développement Python avancé. La conception d'une interface graphique réactive avec `tkinter.ttk`, l'implémentation d'une architecture en couches n-tiers, ainsi que la manipulation sécurisée d'une base SQLite3 avec context managers et clés étrangères (`PRAGMA`) constituent un socle technique solide.

2. Difficultés surmontées

Le principal défi technique a résidé dans la synchronisation de l'affichage graphique Tkinter avec la base de données. La mise en place de callbacks de rafraîchissement (`refresh_table()`) et l'utilisation de `tree.set()` ont permis d'offrir une réactivité instantanée à l'utilisateur.

3. Perspectives d'évolution

Le socle applicatif étant stable et modulaire, plusieurs évolutions sont envisageables à court et moyen terme :

- Module d'administration CRUD pour gérer les ouvertures de comptes directement depuis l'interface.
- Génération et exportation automatisées des relevés de notes semestriels au format PDF ou Word via python-docx.
- Chiffrement local du fichier de base de données .db au repos.

Le code source de l'application est disponible sur GitHub :
HTTPS://GITHUB.COM/HARD2007/PROJET.GIT